



*Software Requirements
Specification*

Smart Society Management

Category: Full-Stack Application Development

Table of Contents

1.1 Background and Necessity for the Web Application	1
1.2 Proposed Solution	1
1.3 Purpose of the Document	1
1.4 Scope of the Project	2
1.5 Constraints	2
1.6 Functional Requirements	2
1.7 Non-Functional Requirements	4
1.8 Interface Requirements	4
1.8.1 Hardware	4
1.8.2 Software	4
1.8.3 Database Design & Data Dictionary	5
1.9 Project Deliverables	5

1.1 Background and Necessity for the Web Application

Modern housing societies and gated communities face substantial administrative challenges in manually managing resident records, security checkpoints, maintenance collection, visitor approvals, and facility bookings. Traditional manual register logs and fragmented communication methods lead to security vulnerabilities, operational delays, untracked financial dues, and poor transparency between society management boards and residents. As residential complexes grow, there is an urgent need for an integrated web application to automate daily operations and improve community security.

1.2 Proposed Solution

The proposed solution titled **SmartSociety** is a comprehensive Web-based Housing Society Management System designed to centralize administration, visitor security, maintenance billing, complaint resolution, and facility booking. The system provides role-based interfaces tailored for Society Administrators, Residents/Homeowners, Security Guards, and Maintenance Staff.

1.3 Purpose of the Document

The purpose of this document is to define the functional and non-functional requirements for the development of the SmartSociety Web application. This document serves as a foundational guide for developers, system architects, and society stakeholders to align software features with operational requirements.

It details the scope, architectural specifications, database schema design, and operational constraints governing the implementation of the society management application.

1.4 Scope of the Project

SmartSociety will be a fully responsive, secure web platform accessible across desktop and mobile browsers.

Residents can view maintenance invoices, log maintenance complaints, request visitor entry passes, book shared amenities (e.g., clubhouse, tennis courts), and participate in digital community voting/notices. Society Administrators can manage resident onboarding, generate automated monthly maintenance bills, oversee visitor logs, assign work tickets to maintenance staff, and broadcast emergency notices.

Security personnel can log incoming guests and delivery vendors, verifying gate passes in real-time. Payment gateway processing and automated banking reconciliation are simulated for scope compliance.

1.5 Constraints

The SmartSociety Web application must comply with the following system constraints:

- **Browser Compatibility:** Seamless operation across modern web browsers, including Chrome, Edge, Safari, and Firefox.
- **Responsiveness:** Mobile-first UI layouts designed for security desk tablets, handheld mobile phones, and desktop screens.
- **Real-time Security Verification:** Gate pass verification and visitor entry approval notifications must execute with minimal latency (< 2 seconds).

1.6 Functional Requirements

For Residents / Flat Owners:

1. **Account Authentication & Resident Profile:** Secure login with multi-factor support. Profile management linking flat numbers, vehicle registrations, emergency contacts, and family/tenant details.
2. **Maintenance Dues & Invoicing:** View current and historical maintenance bills, download PDF receipts, view breakdown of charges (water, security, repairs), and simulate digital fee payments.
3. **Visitor Pre-Approval & Gate Passes:** Generate digital QR gate passes for pre-approved guests, delivery drivers, and cab operators with custom time windows.
4. **Complaint & Helpdesk Portal:** Raise maintenance tickets (e.g., plumbing, electrical, elevator faults) with photo uploads, category tagging, and status tracking (Pending, In-Progress, Resolved).
5. **Facility & Amenity Booking:** Check real-time availability and reserve community amenities (clubhouse, swimming pool, sports courts, party hall).
6. **Notice Board & Digital Polling:** Access official announcements, event calendars, society guidelines, and cast votes on community polls.

For Security Personnel (Gate Management):

1. **Visitor Log Entry:** Record walk-in visitor details, photograph, vehicle number, target flat number, and entry timestamp.
2. **Pass Verification:** Scan/verify resident-generated visitor QR codes or numeric gate keys for instant access clearance.
3. **Overstay & Delivery Alerts:** Receive system alerts for unapproved extended vendor stays inside the society premises.

For Society Administration:

- 1. **Resident & Flat Management:** Onboard/offboard residents, tenants, and owners; maintain flat occupancy status maps.
- 2. **Billing Engine:** Generate monthly maintenance invoices, apply penalties on overdue balances, and monitor collection reports.
- 3. **Helpdesk Routing:** Assign resident complaint tickets to dedicated maintenance personnel and monitor resolution SLAs.
- 4. **Security Supervision:** Access real-time entry/exit logs for all gates, visitors, and staff members.

Common Features & User Journeys:

Includes **Emergency Contact Directory**, **Society Guidelines**, and **Interactive Emergency Siren/Alert**.

- **Resident Journey:** Login → Generate Visitor Pass → View Monthly Bill → Book Clubhouse → Log Plumbing Ticket → Track SLA.
- **Admin Journey:** Login Dashboard → Review Monthly Dues → Broadcast Emergency Notice → Assign Helpdesk Ticket → Audit Security Gate Logs.

1.7 Non-Functional Requirements

Requirement	Description
Data Privacy & Security	Strict access controls protecting resident personal information, phone numbers, and vehicle records.
High Availability	24/7 continuous operation to guarantee uninterrupted gate clearance and emergency alerts.
User-Friendliness	Simplified interface tailored for effortless operation by non-technical security staff and elderly residents.
Performance & Speed	Fast page response times (< 1.5 seconds) for quick security checkpoint validation and pass scanning.
Scalability	Database architecture scalable to support multi-tower complexes with thousands of apartments and daily visitors.
Role-Based Security	Granular privilege management distinguishing Resident, Guard, Staff, and Administrator roles.
Audit Logging	Immutable records for all security gate entries, complaint status changes, and admin financial edits.
Cross-Platform Compatibility	Flawless execution across mobile devices, desktop browsers, and gate-terminal tablets.

1.8 Interface Requirements

1.8.1 Hardware

- Server: Quad-Core Processor, 16 GB RAM, SSD Storage
- Gate Terminals: Android/iOS Tablet or Desktop Workstation with Camera/Barcode Scanner
- Client Devices: Standard Mobile Smartphones or Desktop Computers
- Peripherals: Gate Receipt Printer (Optional)

1.8.2 Software Stack Options

Component	Supported Tech Stack Options
Frontend	HTML5, CSS3, Bootstrap 5, JavaScript, ReactJS / Vue.js / Angular, Tailwind CSS
Backend Options	• Node.js with Express.js framework • C# ASP.NET Core MVC / Web API (.NET 8) • Python 3.10+ with Django / FastAPI • PHP 8+ with Laravel Framework
Database	• MySQL 8.0+, PostgreSQL 14+, or Microsoft SQL Server 2022

1.8.3 Database Design & Data Dictionary

Key entities and attributes defined in the society management schema:

Users user_id (PK) username password_hash role (Admin/Resident/Guard)	Visitors visitor_id (PK) flat_id (FK) visitor_name , phone vehicle_number gate_pass_code	Complaints complaint_id (PK) resident_id (FK) category , description status , created_at
Flats / Units flat_id (PK) block_name flat_number occupancy_type (Owner/Tenant)	MaintenanceBills bill_id (PK) flat_id (FK) amount_due , due_date payment_status	

Key Entity Relationships:

- Flats → Residents: One-to-Many
- Flats → MaintenanceBills: One-to-Many
- Residents → Complaints: One-to-Many
- Flats → Visitors → GateLogs: One-to-Many
- Residents → AmenityBookings: One-to-Many

1.9 Project Deliverables

You will design and build the project and submit it along with a complete documentation that includes:

- Problem Definition
- Design Specifications
- Diagrams such as Flowcharts for various Activities, Data Flow Diagrams, and so on
- Database Design
- Test Data Used in the Project
- Project Installation Instructions
- User Credentials for all Types of Users with Passwords

Documentation is considered as a very important part of the project. Ensure that documentation is complete and comprehensive. However, do not include full source code inside the documentation.

The consolidated project must be submitted as a zip file with a ReadMe.doc file listing assumptions (if any) made at your end and SQL scripts files (.sql) containing database and table definitions.

Note: Preferably, host the working Web application on a Website and share the URL for evaluation.

Submit a video (.mp4 file) demonstrating the working of the full-stack Web application, including all the functionalities of the project. This is MANDATORY.

Sitemap: To help users understand the flow of the Web application, you will create a Sitemap and add it to the home page of your Web application.

Over and above the given specifications, you can apply your creativity and logic to improve the application